Further Lean Proof Simplifications

for ProveIt's DiophantineConsistency development

Six additional proposals after a prior-proposal audit
Prepared for Vladimir Reshetnikov 26 October 2026

Main result. This packet identifies six concrete simplification opportunities that were not found in the two earlier DiophantineConsistency simplification packets indexed by `docs/incoming`, including their supplementary notes. It supplies reviewable patches, independently checkable Lean files, measured source changes, and compiler evidence. The largest immediate benefit is a clearer statement of the mathematical reason for each proof: support classification, an already established Pell parity theorem, an inverse equivalence, uniqueness of Euclidean division, a common arithmetic budget, or transitivity of congruence up to sign.

Five compatible proposals form a conservative patch set with a net reduction of **57 physical production lines**, including all new helper modules and imports. Two of those changes alone save 37 lines without a new helper. The sixth proposal, a reusable sign relation, is an alternative extension: it improves proof structure but introduces enough shared code that the combined net reduction becomes 29 lines. These are modest, measured savings; they are not a claim to have dramatically compressed the whole development.

Strongest verification. The complete modified `PairingRepresentation` module compiled under Lean 4.32.0. Two further proofs establish by `rfl` that its complete inverse-coordinate `GraphRep` values equal the baseline definitions. Their expressions and witness counts therefore remain definitionally unchanged. Other proposals have the narrower validation scopes listed below; no complete repository build or complete downstream rebuild is claimed.

Proposal	Concrete target	Net lines saved
1. Support classification	`sum_le_one_cases`	17
2. Existing Pell parity theorem	Local `hsodd` argument	20
3. Inverse-equivalence projections	Eight inverse-pairing clients	8
4. Shared Euclidean certificate	Two `certificate_iff` proofs	4
5. Affine witness budget	Three `thin_compose` arguments	8
Conservative combination	Includes all helper overhead	**57**
6. Optional sign algebra	Replaces the native/Pell subset above	**29 total**

1 Baseline, scope, and novelty

1.1 A fixed source snapshot

The repository baseline is commit `6175dff9a154`. Its full identifier is recorded in the source manifest and patch metadata. The initial GitHub inspection returned `e73fa2e916c9`; the local checkout then obtained the newer merge used here. That merge preserves both earlier proposal packets and newer maintained proofs. The two files currently in `docs/incoming` have the same Git blob identities at both inspected commits, so this change of baseline does not alter the incoming corpus.

The Lean subtree contains 574 `.lean` files and 77,278 physical lines at this snapshot. This is an inventory, not a claim that every declaration has received a semantic review. The present

work concentrates on six specific repeated arguments. The exact validation environment was Lean 4.32.0, compiler commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`, with mathlib commit `81a5d257c8e410db227a6665ed08f64fea08e997`.

1.2 What was excluded

The audit examined the current archive `diophantine-lean-simplification-2026-10-26.zip` and the earlier placed packet `research/lean-proof-simplification-20261026/`, which is explicitly linked by the incoming README. It included their reports, prototype declarations, patch contents, and source notes. The complete 35-item exclusion matrix is supplied as `novelty-audit.md`.

Previously proposed directions include successful list-lookup APIs, witness block concatenation and assignment transport, primitive-recursive iterator congruence, checked-history folds and scanner laws, common sum-of-squares semantics, topological gate induction, generated exponent certificates, finite dispatch, degree interfaces, half-radix formulas, field-residue geometry, carry decomposition, copied-field bounds, object-model bridges, and closed-sentence transport. None is offered again as a new headline.

One initially promising candidate was explicitly rejected: an additive subgroup presentation of aligned cells. The current incoming report already proposes a later equivalence between aligned cells and `ZMod N`. A new implementation would advance that earlier direction, but it would not meet the user's request for an additional idea.

The novelty claim is deliberately local: these six concrete refactors were not found in the reviewed DiophantineConsistency proposal corpus. It is not a claim of novelty in mathematics or Lean practice. The incoming README also indexes historical Busy Beaver packets. Their visible summaries informed the exclusions, but their hundreds of ancillary files were not all opened. The audit records this limit rather than claiming an exhaustive history of every earlier research submission.

2 Use the existing finite-support classification

2.1 The redundant argument

In `CurrentCellLabels2011.lean:131`, the theorem `sum_le_one_cases` classifies a natural-valued function whose finite sum is at most one. The current proof selects a nonzero term, proves it equals one, erases its index from the sum, proves the remaining sum vanishes, and then proves every other coordinate vanishes. All of those steps express one support property already available in mathlib.

The existing theorem `Finset.sum_le_one_iff` states, in the present natural-number specialization, that if the sum is at most one, any two nonzero summands have the same index and the selected value is one. Its conclusion is almost exactly the repository's existing `Selected` predicate:

$$\mathrm{Selected}(f, i) \quad \Longleftrightarrow \quad f(i) = 1 \ \wedge \ \forall j \neq i, \ f(j) = 0.$$

2.2 The replacement

After choosing a nonzero coordinate only when one exists, the replacement is:

```
  classical
  have h := Finset.sum_le_one_iff.mp hf
  by_cases hz : ∀ i, f i = 0
  · exact Or.inl hz
  · obtain ⟨i, hi⟩ := not_forall.mp hz
    refine Or.inr ⟨i, (h i i (by simp) (by simp) hi hi).2, ?_⟩
    intro j hji
    by_contra hj
    exact hji (h j i (by simp) (by simp) hj hi).1
```

The proof exposes the actual invariant directly: the support is empty or a singleton carrying the value one. It preserves the exact theorem statement and the existing definition of `Selected`. In particular it requires neither a default element nor a `Nonempty` instance. If the index type is empty, the first branch is valid. The natural-valued assumption is material: allowing arbitrary signed values would permit cancellation and would invalidate this support conclusion.

The whole production file changes from 311 to 294 physical lines, a saving of 17. The exact theorem statement and exact `Selected` definition compile in `native-proofs/OneHotSupport.lean`. The complete `CurrentCellLabels2011` dependency cone was not rebuilt. The patch therefore has a checked replacement theorem, rather than a claim of full module integration.

Why this adds something new. The earlier finite-dispatch and copied-field proposals concern data assembly and field bounds. They do not replace the mathematical support-uniqueness argument in this theorem.

3 Reuse the already proved Pell parity criterion

3.1 Two proofs of the same number-theoretic fact

The theorem `index_residues` in `NativePellStepDown.lean:17` contains a local proof that an auxiliary Pell index `s` is odd. It argues by contradiction: an even-index first Pell coordinate has residue a power of minus one modulo the Pell parameter; divisibility would then force the parameter to divide one, contradicting its lower bound.

The repository already provides precisely the needed fact in `SignedPellResidue.lean:42`:

```
odd_index_of_base_dvd
  (hA : 1 < A) (hd : A | xn hA n) : Odd n
```

The local Pell classification equation supplies its divisibility premise. Consequently the replacement is:

```
  have hsodd : Odd s :=
    SignedPellResidue.odd_index_of_base_dvd hI1 (by
      rw [← hIu]
      exact dvd_mul_right I u)
```

The now unused local proof `hs0 : 0 < s` is also removed. The public theorem's hypotheses are retained; this patch does not require editing its callers. Including the additional import, the file changes from 248 to 228 lines, a net saving of 20 without a new helper module.

3.2 Integration and novelty boundaries

`SignedPellResidue` imports the independent Pell integer development; it does not import `NativePellStepDown`. The source dependency inspection found no cycle introduced by the proposed import. The patch applies to the pinned source, but the entire modified Pell module was not compiled in this session. This is the most important remaining validation step for this otherwise small change.

The separate optional sign abstraction in Section 7 is not required for this saving. The file `native-proofs/pell-oddness-only.patch` is the independent minimal version. It is an alternative to the larger native combined patch, not an additional patch to apply after it.

The earlier proposal to decode Pell register equations into arithmetic facts is different. Here the relevant arithmetic fact has already been proved in another maintained source file; the improvement is to reuse that exact theorem at an overlooked client.

4 Derive inverse graphs from a product equivalence

4.1 A representation-independent theorem

Eight inverse-pairing clients currently re-create a missing coordinate, apply a pairing correctness theorem, and then use the two pairing/unpairing identities. This is the inverse-image property of an equivalence.

For any equivalence $e : A \times B \simeq C$, any $z : C$, and any $a : A$,

$$(\exists b : B,\ z = e(a, b)) \quad \Longleftrightarrow \quad a = (e^{-1}z)_1.$$

The corresponding second-coordinate theorem is symmetric. Applying e^{-1} converts the equation into equality of ordered pairs; existentially quantifying the unused coordinate then leaves precisely the retained projection. This needs no encoding-specific arithmetic and no choice of a fresh witness beyond the coordinate already supplied by $e^{-1}z$.

Both general lemmas fit in a 16-line production module:

```
theorem exists_right_iff {A B C : Type*}
    (e : A × B ≃ C) (z : C) (a : A) :
    (∃ b, z = e (a, b)) ↔ a = (e.symm z).1 := by
  simp only [← e.symm_apply_eq, Prod.ext_iff]
  simp [eq_comm]
```

Mathlib already supplies `Nat.pairEquiv`. The repository's inverse-pairing proofs can specialize the general lemma instead of proving the inverse-image argument afresh.

4.2 Separate tuple normalization from inverse reasoning

For the one-witness client in `PairingRepresentation.lean:47`, the proof after its existing `change` statement becomes:

```
simp only [Fin.exists_fin_succ_pi, Fin.exists_fin_zero_pi,
  Fin.cons_zero]
exact PairingProjection.exists_right_iff
  Nat.pairEquiv (v (some 0)) (v none)
```

The first step is only a normalization of the existing one-element witness tuple. The second step is the new semantic abstraction. This distinction matters for novelty: previous packets proposed general witness layouts, but they did not derive arithmetic inverse graphs from product equivalences.

For a quadratic certificate, the normal form is

$$\exists b, t, s, m,\ \text{Certificate}(z, a, b, t, s, m).$$

The existing `exists_certificate_iff` eliminates t, s, m, leaving $\exists b, z = \text{pair}(a, b)$. The same projection lemma finishes the proof. The tree-code version first eliminates the equality fixing the predecessor coordinate and then follows this identical semantic argument. This preserves the supplied certificate data and does not change how the register program or polynomial is constructed.

The eight clients are distributed across four modules. The direct and quadratic representations appear in `PairingRepresentation.lean:47` and `QuadraticNatPairing.lean:104`. The register and tree-code clients appear in `RegisterNatPairing.lean:92` and `QuadraticTreeCodeGraphs.lean:77`. The four files lose 24 lines after their added imports; the new helper adds 16, giving a net saving of eight.

4.3 A full module check and a data-preservation proof

The baseline `Representation`, `ArithmeticRepresentation`, `ComposeRepresentation`, and `PairingRepresentation` modules were compiled. The new helper and the complete modified `PairingRepresentation` were then compiled. Finally the original two inverse-coordinate definitions were copied, with changed names, into `PairingDataAudit.lean`. Lean accepted:

```
example : unpairLeftCoordinate = unpairLeftCoordinateBaseline := rfl
example : unpairRightCoordinate = unpairRightCoordinateBaseline := rfl
```

These compare complete `GraphRep` values. Their data fields are definitionally the same, and proof irrelevance accounts for the changed correctness proof. Both witness counts remain three. This is stronger evidence than observing that the patch visually changes only proof text.

The other six clients are supplied as source-reviewed candidates. The exact quadratic and tree witness-obligation shapes are checked separately under their existing certificate specification; this does not substitute for compiling their complete production modules. A small elaboration issue was also resolved during the full check: `simpa ... using` left the generic equivalence coercion folded, while `simp only ...` followed by `exact` allowed definitional equality to close the goal. The delivered patch contains the successful form.

5 Share the positive-denominator Euclidean certificate

5.1 An exact duplicate across two backends

`RegisterDivMod.lean:9` and `QuadraticDivMod.lean:10` contain the same 17-line theorem. Both characterize a certificate with one natural slack:

$$\exists s \in \mathbb{N},\quad n = qb + r \ \wedge\ b = r + s + 1 \quad\Longleftrightarrow\quad b > 0 \ \wedge\ q = \lfloor n/b \rfloor \ \wedge\ r = n \bmod b.$$

The register and quadratic encodings are different, but this arithmetic lemma is independent of both. Importing one full backend into the other would introduce an unnecessary representation dependency. A small `ArithmeticCertificates` module provides an appropriate common home.

The new proof uses Lean's existing `Nat.div_mod_unique`. The forward direction extracts $0 < b$ and $r < b$ from $b = r + s + 1$, then applies quotient/remainder uniqueness once. The reverse direction uses the usual division decomposition and the canonical slack $b - (n \bmod b) - 1$. The production helper imports only `Std.Tactic`; it does not require either representation compiler.

Both public declarations retain their original statement and become:

```
theorem certificate_iff (q r n b : ℕ) :
    (∃ s, n = q * b + r ∧ b = r + s + 1) ↔
      0 < b ∧ q = n / b ∧ r = n % b :=
  ArithmeticCertificates.divMod_certificate_iff q r n b
```

5.2 The guard and a useful uniqueness extension

The positive-denominator guard must remain part of the represented predicate. Lean's natural division is total at zero, but a zero denominator cannot satisfy $b = r + s + 1$. Dropping the guard would therefore change the certificate's zero set. The supplied check file proves explicitly that no certificate exists with denominator zero.

It also proves a stronger fact useful for subsequent refactors: two certificates for the same n, b determine the same quotient, remainder, *and slack*. Quotient/remainder uniqueness gives $q = q'$ and $r = r'$; the two equations $b = r + s + 1 = r' + s' + 1$ then give $s = s'$. The actual checked conclusion is `q = q' ∧ r = r' ∧ s = s'`. This can support future proofs by uniqueness rather than repeated explicit witness recovery.

The complete new production helper is 20 lines. The two backend files together change from 159 to 135 lines, so including the helper the net change is 159 to 155, a saving of four. The primary gain is one maintained semantic proof and independent backend dependencies. All source bytes outside the two changed declarations and imports were checked to be unchanged.

The helper, exact public wrapper type, zero-denominator result, canonical certificate result, and uniqueness extension all compiled. The full backend modules were not rebuilt. The logged axiom probes for these checks contain only `propext` and `Quot.sound`.

Novelty boundary. Earlier carry-normalization proposals concern pointwise quotient/digit decompositions. They do not identify or share this exact existential positive-denominator certificate across the two backends.

6 Factor the arithmetic budget behind composition

6.1 The repeated argument

The three composition theorems in `GraphSizeBounds.lean:48` (at lines 48, 113, and 132) repeat the same arithmetic proof around different semantic depth/count laws. The invariant is

$$\mathrm{Thin}(C, r): \quad \mathrm{depth}(r) \le 6\,\mathrm{witnessCount}(r) + C.$$

Each proof takes the total of the parent and child witness counts, bounds every child by that total, invokes the appropriate composition-depth rule, and pays for the fixed composition overhead with additional witnesses.

This can be stated as an arithmetic lemma, leaving the source expressions and graph constructors entirely out of its definition. Let

$$T = w_0 + \sum_i w_i, \qquad d_0 \le 6w_0 + C, \qquad d_i \le 6w_i + C, \qquad C \ge 1.$$

Suppose a constructor rule gives $D \le h + o$ whenever the parent and child depths are at most h, and suppose $o \le 6k$ and $k + T \le W$. Then

$$D \le (6T + C) + o \le 6(T + k) + C \le 6W + C.$$

The only finite-sum fact needed is $w_i \le \sum_j w_j$, valid because the weights are natural numbers. The only purpose of $C \ge 1$ here is to make the common bound $6T + C$ admissible to the existing depth rule.

6.2 The small production interface

The 21-line production helper accepts the depth rule as a proof parameter, so it can be used by three existing constructors without reimplementing their semantic proofs. One of the new client bodies is:

```
exact AffineBudget.compose
  (fun i => (s i).witnessCount) (fun i => (s i).depth)
  hC hr hs
  (fun _ hh hr hs => compose_depth_le r s hh hr hs)
  (by omega : 2 * n + 2 ≤ 6 * n)
  (by rw [BoundedForallSize.predicate_compose_count]; omega)
```

The production patch retains the established coefficient six rather than introducing a more general resource framework with no present clients. The three original proof bodies are replaced by applications of the same lemma. `GraphSizeBounds` changes from 214 to 185 lines; after adding the 21-line helper, the net saving is eight.

6.3 An unnecessary premise becomes visible

For predicate composition and direct graph composition, the current overhead is $2n+2$, with n additional witnesses. The budget inequality $2n + 2 \le 6n$ requires $n \ge 1$. For ordinary graph composition, the overhead is $2n + 4$, while $n + 1$ additional witnesses are available:

$$2n + 4 \le 6(n + 1) \qquad \text{for every } n \in \mathbb{N}.$$

Thus the arithmetic argument for ordinary graph composition does not need the current `hn : 1 ≤ n` hypothesis. The check module proves this zero-arity-inclusive numerical rule. The delivered production patch keeps the old public signature to avoid a caller migration, but a follow-up generalized theorem can remove this premise after a full module check. The direct-composition case must retain its different restriction.

The helper and both numerical application regimes compiled. Their hypotheses are precisely the metadata supplied by the existing count/depth theorems. The complete `GraphSizeBounds` module was not rebuilt. This is a checked arithmetic core plus a source-reviewed client patch.

Further development. The long constructor-bound proof in `BoundedForallDepth` is a separate next client worth investigating: its repeated maximum and offset calculations could be replayed through a small max-plus bound language. No such language is implemented or counted in this packet. Unlike the earlier syntax-degree proposal, this direction concerns depth and witness budgets, including their arity-dependent overhead.

7 Make congruence up to sign compositional

7.1 The mathematical relation

Several final Pell arguments carry a residue only up to a sign. Working directly with disjunctions such as $m \mid a-b$ or $m \mid a+b$ creates a four-branch sign chase whenever two such facts are combined. Introduce the relation

$$a \sim_m b \quad \Longleftrightarrow \quad a \equiv b \pmod{m} \ \vee \ a \equiv -b \pmod{m}.$$

It is symmetric and transitive. It also has the divisibility form $m \mid a - b \vee m \mid a + b$, is unchanged by multiplying one endpoint by 1 or -1, and relates an integer to the natural cast of its absolute value. The definition and proofs work for every integer modulus, including zero; no positivity hypothesis is hidden in the interface.

The implementation uses `Int.ModEq` and its existing negation and transitivity lemmas. The unavoidable four-way sign case split is proved once in the relation's transitivity theorem. Clients then compose facts at the level at which they reason mathematically.

7.2 The Pell client

The final part of `NativePellStepDown.index_residues` can be represented by the following chain:

$$J \sim_m u \sim_m ts \sim_m s \sim_m p, \qquad t \in \{1, -1\}.$$

The supplied theorem `signed_index_chase` proves this chain once and converts its result back into the existing divisibility disjunction. The client becomes:

```
exact ProofSimplification.signed_index_chase hu_J hu_cc
  (neg_one_pow_eq_or ℤ h) hsp'
```

The same interface shortens absolute-value transport in `SignedPellResidue`. The generic helper and both application theorems compile; the complete modified Pell modules remain to be rebuilt.

7.3 Do not hide the abstraction cost

The readable helper, its application lemmas, comments, and module scaffolding occupy 64 physical lines. With the one-hot, parity, and sign changes together, the three existing native files lose 73 lines; after the new helper, the native subset saves only nine. The simpler one-hot plus parity subset saves 37. Therefore adding the sign API costs 28 lines relative to that minimal subset. Across all proposals, the conservative 57-line saving becomes 29.

The case for the sign API is semantic structure and future reuse, not a claim of maximal immediate line compression. Adopt the minimal parity patch first if current source size is the priority; consider the sign API when further clients need the same relation. No time or object-size speedup is inferred from the smaller client proof.

Novelty boundary. Earlier proposals to decode the Pell source equations do not supply this sign relation, transitivity, or absolute-value transport. This is a different proof interface applied after the arithmetic facts have been obtained.

8 Validation and reproducibility

8.1 What actually ran

Artifact	Observed result and scope
`OneHotSupport`	Exact replacement statement and `Selected` definition compiled. Full current-cell module not rebuilt.
Pell parity-only patch	Applies to the pinned source; existing theorem and dependency inspected. Full patched Pell theorem not compiled.
`PairingProjection`	Both general inverse-projection lemmas compiled.
Modified `PairingRepresentation`	Complete module compiled after its three baseline dependencies.
`PairingDataAudit`	Both complete inverse `GraphRep` equalities proved by `rfl`; witness counts remain three.
Quadratic/tree projection checks	Schematic exact witness-obligation shapes compiled under the existing certificate specification. Other full production clients not rebuilt.
`ArithmeticCertificates`	Helper, exact wrapper type, positive canonical witness, zero-denominator exclusion, and full certificate uniqueness compiled.
`AffineBudgetChecks`	Production helper and direct/ordinary numerical client rules compiled, including the ordinary zero-arity case.
`SignedModEq`	Relation laws and sign-chase/absolute-value application theorems compiled. Full Pell clients not rebuilt.
All supplied patch combinations	`git apply --check` performed against the fixed baseline. This is a textual integration check, not a Lean build.

New checked files contain no `sorry`, `admit`, or added axiom declarations. The recorded axiom probes list only the usual `propext`, `Classical.choice`, and `Quot.sound`, with the Euclidean certificate checks using only the first and last. These checks say nothing new about the repository's global consistency endpoint or its complete dependency assumptions; they concern the specified local theorems.

Measured compiler durations are retained as execution evidence, not presented as performance benchmarks. They are single observations from a prepared environment. A whole-module timing comparison, proof-object size study, and downstream regression build would be separate experiments.

8.2 Patch choices

The top-level `patches/conservative.patch` combines support classification, parity reuse, pairing projections, arithmetic certificates, and affine budgets. The alternative `patches/with-sign-algebra.patch` replaces the native subset with the larger sign-interface version. Apply one aggregate patch, not both. Individual patches are also retained for separate reviews. No change was committed or pushed to GitHub during this work.

The archive includes complete candidate modules, so the proposed result can be inspected without applying a patch. The candidate files preserve all unrelated baseline contents. Scripts regenerate patch text and count lines; the shared replay script runs the isolated checks in an existing Lean 4.32.0 project, and has an option to compile the complete pairing example in a temporary output directory. It does not install tools or claim to run the whole repository build.

The full repository should still be built in the normal project environment before adoption. In particular, the simple-looking Pell import reuse is not upgraded to "compiled" merely because its invoked theorem exists. The distinction is retained in both the human report and JSON evidence.

9 Further research questions

1. Can the inverse-equivalence lemma replace analogous missing-coordinate proofs for the repository's Cantor encoding, while preserving each encoding's actual equivalence and avoiding any accidental identification with `Nat.pair`? The abstraction permits this, but an additional concrete client audit is needed.
2. Can certificate uniqueness be used to derive coherence of composed quotient/remainder witnesses, eliminating repeated reconstruction arguments? The checked three-coordinate uniqueness result is a suitable starting point; a claim of savings requires identifying additional production clients.
3. Can ordinary graph composition expose a theorem without its nonempty arity premise? The numerical obstacle has been removed here. The stronger production theorem should be checked with the existing count/depth API and then introduced without disrupting positional callers.
4. Can a small certified max-plus calculation replace the manually named intermediate depth bounds in `BoundedForallDepth`, with the old semantic constructor lemmas as its leaves? The challenge is to avoid expanding large representation expressions during elaboration.
5. At how many additional Pell clients does the sign relation become a net source reduction relative to direct case analysis? Count the complete helper and compatibility wrappers, and distinguish readability from run time.
6. Would a declaration-level search for proofs already subsumed by existing theorems find further parity-style wins? The useful target is a precise mathematical implication, not indiscriminate rewriting of every proof into generic automation.

The recommended first review is the 37-line helper-free support/parity subset, followed by the fully compiled pairing example. The remaining small shared interfaces are concrete follow-on changes with explicitly recorded build gaps. Their value comes from making repeated mathematical obligations recognizable and maintaining them once.

Primary sources and artifact map

1. ProveIt baseline `6175dff9a154`. Source links throughout this report are pinned to this commit.
2. Incoming directory and the two DiophantineConsistency packets identified in Section 1. Detailed exclusion evidence: `novelty-audit.md`.
3. mathlib `81a5d257c8e410db227a6665ed08f64fea08e997`: finite-sum support theorem, natural pairing equivalence, `Int.ModEq`, finite tuples, and equivalence identities.
4. Lean `8c9756b28d64dab099da31a4c09229a9e6a2ef35`: exact 4.32.0 compiler used for the recorded checks.
5. `evidence/validation-summary.json`, execution logs, source manifest, and production metrics: machine-readable support for the validation and line-count claims.
6. `native-proofs/`, `certificates/`, and `projection-and-budget/`: proposal code, exact patches, full candidate modules, and reproduction scripts. `scripts/replay.py` provides the common check entry point.